

ClaudeOS

AI Operating System — Client Handbook

Prepared for: faiyke-ai

Version: 13.1 · 2026-05-24

Confidential — For authorised users of ClaudeOS only.
Do not distribute outside your organisation.

Contents

#	Section	What You Will Find
1	Introduction	What ClaudeOS Is and What It Does
2	System Requirements	Hardware, Software, and Network
3	Installation & First-Run Setup	From Zero to Running in 15 Minutes
4	Logging In	Authentication, Roles, and Password Rules
5	Dashboard Tour	Every Page Explained
6	Working with AI Agents	Chat, Catalog, Run History
7	Memory System	How ClaudeOS Remembers Things
8	Workflows & Automation	Pipelines, Scheduling, Webhooks
9	Client Vault	Namespaces, Projects, Context Files
10	Outputs & Reports	Saving, Searching, Exporting
11	Ticketing System	Support and Task Lifecycle
12	Quality Scoring	LLM-as-Judge Automatic Evaluation

13	Observability	Performance, Latency, Token Cost
14	Advanced Features	Images, Voice, MCP, A2A
15	API Reference	REST API for Developers
16	Security Model	Auth, Isolation, Audit Log
17	Cloud Sync	Supabase Backup
18	Troubleshooting	Common Problems and Fixes
19	Support	How to Get Help

1 — Introduction

What Is ClaudeOS?

ClaudeOS is an AI Operating System — a secure, multi-user coordination layer that brings together AI agents, a long-term memory engine, automated workflows, project management, and a full ticketing system into one authenticated control centre. It runs as a private web application on your own infrastructure, so your data never leaves your environment.

Think of it as a team of 12 specialist AI assistants, all sharing the same memory, working from the same context, and all governed by the same security and quality rules — accessible from any browser.

Core Capabilities

Capability	What It Does for You
AI Agents	12 specialist agents covering research, writing, analysis, scheduling, QA, client management, comms, and more. Run them via chat, one-shot dispatch, or automated pipeline.
Long-Term Memory	Every fact, preference, reminder, and task you store is available to every agent. Memory persists between sessions and is scoped per namespace so each client or project only sees its own data.
Workflow Automation	7 built-in pipelines (morning briefing, memory curation, research digest, client report, analysis run, QA sweep, meta-orchestration) that run on a schedule or on demand.
Ticketing	Full support and task lifecycle — create, assign, prioritise, comment, resolve, and bulk-manage tickets without leaving ClaudeOS.
Quality Scoring	Every agent response is automatically scored on 4 dimensions by a fast AI judge. Low-quality runs are flagged before they reach your workflow.

Observability

Real-time dashboards for response quality, latency, token spend, and memory health — so you always know how the system is performing.

Architecture at a Glance

Browser ■■■ Streamlit Dashboard (:8501) ■ Login Gate (JWT) ■ Flask REST API (:5000) ■■ Auth & Roles ■■ Agent Dispatcher ■■ Memory Engine (SQLite FTS5 + ChromaDB) ■■ Workflow Scheduler (APScheduler) ■■ Output Manager ■■ Ticketing ■■ Observability Optional: MCP Tool Server (:5100) – exposes agents to external AI clients

Note: ClaudeOS runs entirely on your own server. No agent prompts, memory entries, or outputs are sent to any third party except the Anthropic Claude API (for agent inference) and optionally Supabase (for cloud backup, if you choose to enable it).

2 — System Requirements

Server (where ClaudeOS runs)

Component	Minimum	Recommended
OS	Windows 10 64-bit	Windows 10/11 64-bit
CPU	2 cores	4+ cores
RAM	4 GB	8 GB+
Disk	10 GB free	20 GB+ SSD
Python	3.11	3.11 or 3.12
Internet	Required for Claude API calls	Stable broadband

Client Devices (browsers)

Any modern browser (Chrome, Edge, Firefox, Safari). No installation required on client devices. Mobile browsers are supported but a desktop screen is recommended for the Observability and Admin panels.

Required Accounts & Keys

Service	Purpose	Cost
Anthropic Claude API	Powers all AI agent inference. Required.	Pay-per-token (see anthropic.com/pricing)
Supabase (optional)	Cloud backup of memory and outputs.	Free tier available

Important: Keep your `ANTHROPIC_API_KEY` secure. Never share it. If it is compromised, rotate it immediately in the Anthropic console and update your `.env` file.

3 — Installation & First-Run Setup

Step 1 — Get the Code

Copy the ClaudeOS project folder to your server. You should have a folder named `claudeos` with the following top-level layout:

```
ClaudeOS/ ■■ agents/ ■■ core/ ■■ dashboard/ ■■ docs/ ■■ memory/ ■■ mcp/ ■■ scripts/ ■■  
workflows/ ■■ requirements.txt ■■ .env.example
```

Step 2 — Create Your Environment File

Copy `.env.example` to `.env` and fill in your values:

```
# .env ANTHROPIC_API_KEY=sk-ant-... # required FLASK_SECRET_KEY=change-this-secret # any long  
random string FLASK_PORT=5000 STREAMLIT_PORT=8501 # Optional – Supabase cloud backup  
SUPABASE_URL=https://your-project.supabase.co SUPABASE_SERVICE_KEY=your_service_role_key
```

Never commit `.env` to version control. It contains secrets. The `.gitignore` already excludes it.

Step 3 — Install Python Dependencies

```
pip install -r requirements.txt
```

This installs Flask, Streamlit, the Anthropic SDK, ChromaDB, sentence-transformers, APScheduler, waitress, plotly, PyJWT, bcrypt, and all other required packages.

For optional features:

```
# Voice input (downloads ~140 MB model on first use) pip install openai-whisper # MCP Tool  
Server (for external AI client integration) pip install mcp uvicorn
```

Step 4 — Initialise the Database

```
python scripts/migrate.py
```


Creates `data/claudeos.db` with all tables including auth, memory, agents, workflows, tickets, and observability columns.

Step 5 — Seed Default Data

```
python scripts/seed_agents.py # loads 12 agent definitions python scripts/seed_workflows.py #  
loads 7 default pipelines python scripts/seed_namespaces.py # creates the global namespace
```

Step 6 — Create the Admin Account

```
python scripts/create_admin.py --username admin --password Admin123!
```

Use a strong password. Password rules: minimum 10 characters, at least one uppercase, one lowercase, one digit.

Step 7 — Start the System

```
.\scripts\start.ps1
```

This script kills any existing processes on ports 5000 and 8501, starts the Flask API via waitress, verifies `/health`, then starts the Streamlit dashboard.

After 5–10 seconds, open your browser to **[<http://YOUR-SERVER-IP:8501>]** — you should see the ClaudeOS login screen.

Optional — Start the MCP Server

```
.\scripts\start_mcp.ps1
```

Starts the MCP Tool Server on port 5100. Only needed if you want to connect Claude Desktop or other MCP-compatible AI clients to ClaudeOS agents.

Stopping the System

Close the PowerShell windows running the API and dashboard, or kill the processes on ports 5000 and 8501:

```
netstat -ano | findstr :5000 # get PID taskkill /F /PID <PID> # kill it # repeat for 5001, 8501
```

After any change to .env: always restart the server. Running processes do not pick up environment changes automatically.

4 — Logging In

First Login

Navigate to the dashboard URL in your browser. You will see the ClaudeOS login screen with two tabs: **Login** and **Register**.

Enter the username and temporary password provided by your administrator, then press **Enter** or click **Login**. If the admin set the *must change password* flag on your account, you will be prompted to set a new password immediately before continuing.

Password Requirements

- Minimum 10 characters
- At least one uppercase letter (A–Z)
- At least one lowercase letter (a–z)
- At least one digit (0–9)

Self-Registration (if enabled)

If your admin has enabled self-registration:

1. Click the **Register** tab on the login screen
2. Enter username, email, password, and select your namespace
3. Click **Register** — you are returned to the Login tab
4. Log in with your new credentials

User Roles

Role	What They Can Do
Admin	Full access — all namespaces, user management, admin panel, all features.

Operator	All namespaces and features except user management and the Admin panel.
Client	Own namespace only — memory, agents, outputs, tickets. Cannot manage workflows or see the Observability dashboard.
Viewer	Own namespace, read-only — cannot write memory or run agents.
Staff	Support role — only sees tickets assigned to them. Can self-assign open tickets and advance status.

Account Security

- **Account lockout:** 5 failed login attempts triggers a 15-minute lockout. Ask your admin to unlock if needed.
- **Session tokens:** JWT access tokens expire after 60 minutes and are silently refreshed while you are active. Closing the browser logs you out.
- **Case-insensitive usernames:** Alice, ALICE, and alice all refer to the same account.

Theme Toggle

A dark/light mode button sits in the **bottom-left corner** of every page, including the login screen. Click it to switch themes. Your preference persists for the current session.

5 — Dashboard Tour

Overview Page

The home page. Shows the current health of the system and recent activity at a glance.

Element	What It Shows
System Status strip	Green/red indicators for the API and database.
KPI Cards	Total memory entries, registered agents, total runs, workflows, and outputs.
Recent Events	Last 8 agent run results with agent name, namespace, timestamp, and quality score pill.
Quick Dispatch	Run any agent against any namespace with a one-off prompt — no need to open the Agents page.
Memory by Namespace	Count of active memory entries per namespace.
Auto-refresh toggle	When enabled the feed refreshes every 8 seconds — useful for monitoring live runs.
Error alert strip	Red banner shown when any recent run failed. Clears when errors are gone.
Running-now indicator	Yellow banner listing agents currently executing.

Eval Score Pills

Each run entry in the Recent Events feed shows a colour-coded quality pill:

Colour	Score Range	Meaning
--------	-------------	---------

 Green	4.0 – 5.0	High quality — output addressed the prompt well.
 Amber	2.5 – 3.9	Acceptable — review if output is going to a client.
 Red	0 – 2.4	Low quality — re-run with a more specific prompt.

Memory Page

Browse, search, write, and delete memory entries.

- Full-text search across keys and values
- Filter by namespace, category (fact / preference / reminder / task), and minimum confidence
- Write entries with category, TTL, and confidence score
- Reminder entries use a date/time picker for scheduled expiry
- Toggle **Hybrid Search** for BM25 + vector semantic retrieval (higher recall, slightly slower)
- Bulk delete: select entries with checkboxes, then use the bulk toolbar

Agents Page

Three tabs — Chat, Catalog, and Run History.

Chat Tab

Conversational multi-turn interface. Select an agent and namespace, type your message, press Enter. Responses stream token-by-token as they arrive.

- **Image upload:** attach a screenshot, chart, or document for visual analysis
- **Voice input:** click the microphone, record, auto-transcribed into the prompt field (requires Whisper)
- **Conversation history:** full back-and-forth shown per agent per session
- **Clear conversation:** resets history and starts a new session
- **Eval badge:** appears below each response ~10 seconds after completion
- **Token count:** shown after each response

Catalog Tab

Grid of all registered agents with descriptions and a one-shot run form. Good for quick single-task dispatches where you do not need conversation history.

Run History Tab

Complete record of every run across all agents:

- Quality score pill per run
- Expandable eval dimension breakdown (task completion, factual grounding, conciseness, safety)
- Per-agent average score in the summary header
- Bulk delete via checkbox selection

Workflows Page

See all 7 configured pipelines, trigger them manually, and monitor run history. The **Webhooks** tab lets you generate a secret URL so external systems can fire a workflow with a single HTTP POST — no login required.

Client Vault Page

- **Namespaces:** view your namespace details, workspace stats (files, KB), project count
- **Projects:** list projects in your namespace, update status (active / paused / archived)
- **Context Files:** upload files that are automatically injected into every agent system prompt for your namespace — ideal for brand guidelines, SOPs, or client briefs

Outputs Page

All AI-generated content saved by agents — reports, summaries, plans, analyses, code. Full-text search, date and type filters, and per-output export to Markdown or PDF. Bulk delete via checkbox selection.

Tickets Page

Full support and task lifecycle. Covered in detail in Section 11.

Observability Page (Admin / Operator)

Four sub-tabs: Quality Scores, Latency, Token Cost, Memory Health. Covered in Section 13.

Settings Page (Admin / Operator)

System configuration, Supabase sync controls, environment and runtime info.

Admin Panel (Admin only)

User management, API key management, audit log, session management, and security configuration. Covered in Section 16.

6 — Working with AI Agents

The 12 Built-In Agents

Agent	Category	What It Does
Analysis Agent	Analysis	Data analysis — structured reports, key findings, and recommendations from raw data or context.
Morning Briefing Agent	Ops	Synthesises overnight context into a structured morning briefing: news, reminders, client updates, priority actions.
Client Manager Agent	Ops	Per-client project status cards with deadlines, blockers, and action items.
Communications Agent	Comms	Client-facing emails, messages, and proposals — tone-matched to each client.
Memory Curator Agent	System	Reviews, deduplicates, and consolidates memory entries to keep the store lean and coherent.
Meta Orchestrator Agent	System	Decomposes complex goals into ordered chains of agent invocations.
QA Agent	Engineering	Code review against OWASP, platform compatibility, and brand compliance rules.
Research Agent	Research	Deep research with synthesis and source validation across a given topic.
Scheduling Agent	Ops	Converts natural language scheduling requests into memory reminder entries with expiry timestamps.
Transport Ops Agent	Domain	Domain-specific fleet and booking analysis — namespace-locked to its assigned project.
Workflow Builder Agent	System	Converts plain-English automation descriptions into ClaudeOS workflow YAML definitions.

Writing Agent

Content

Drafts reports, emails, and proposals with voice matched to the active namespace.

Running an Agent — Three Ways

1. Chat (Recommended for iterative work)

Go to **Agents** → **Chat**. Select the agent and namespace from the dropdowns, type your request, and press Enter. The response streams back in real time. You can follow up with additional messages in the same conversation.

Best practice: be specific. Instead of "write a report", say "write a 2-page executive summary of the Q2 analysis results for the mobile app project, highlighting the top 3 risks". Specific prompts score higher on eval.

2. Catalog (For one-shot tasks)

Go to **Agents** → **Catalog**. Find the agent, fill in the prompt and namespace, click **Run**. The run is polled every 3 seconds until complete.

3. Quick Dispatch (From Overview)

The Overview page has a Quick Dispatch widget — select agent, namespace, enter prompt, run. Good for fast one-off tasks without navigating away from the dashboard.

Namespace Scoping

Every agent run is scoped to a *namespace*. The namespace controls which memory entries are injected into the agent's context and where the output is stored. If you are an admin or operator you can select any namespace. If you are a client you only see your own.

Image Analysis

In the Chat tab, use the image uploader below the prompt field to attach a file. The agent receives the image as a content block alongside your prompt. Useful for:

- Analysing screenshots of dashboards or reports
- Reviewing UI designs or wireframes

- Extracting data from charts or tables in images

Voice Input

Click the microphone button in the Chat tab. Record your request. ClaudeOS transcribes the audio locally using Whisper and populates the prompt field. Review the transcription, edit if needed, then submit as normal. Requires `pip install openai-whisper`.

Understanding Token Usage

Each agent response shows input tokens (context sent) and output tokens (response generated). Tokens drive API cost. To keep costs low:

- Use specific prompts (less context padding needed)
- Clear conversation history when starting a new topic
- Use lighter agents (e.g. Scheduling Agent) for simple tasks

7 — Memory System

What Is Memory?

Memory is ClaudeOS's persistent knowledge store. When you or an agent writes a memory entry, it is stored in the database and automatically injected into every subsequent agent run in that namespace. This means agents remember facts, preferences, context, and reminders across sessions — without you having to repeat yourself.

Memory Categories

Category	Use For	Example
Fact	Stable information about a client, project, or domain	"Client prefers invoices in GBP"
Preference	Tone, style, or format preferences	"Reports should start with an executive summary"
Reminder	Time-sensitive items that expire automatically	"Follow up on proposal by 2026-06-01"
Task	Action items for agents or team members	"Prepare Q3 analysis before board meeting"

Writing a Memory Entry

1. Go to **Memory** page
2. Click **Add Entry**
3. Fill in: key (short identifier), value (the content), category, TTL (optional expiry in days), confidence score (0–1)
4. For reminders: use the date/time picker to set the exact expiry
5. Click **Save**

Searching Memory

The search bar at the top of the Memory page performs full-text search across keys and values. Toggle **Hybrid Search** for BM25 + semantic vector retrieval — better recall for conceptual queries (e.g. "client payment terms" finds entries about invoicing even if they do not contain those exact words).

How Memory Is Injected into Agents

When an agent runs, ClaudeOS retrieves the most relevant memory entries for the current query using hybrid search, then injects them into the agent's system prompt as structured context. This is done automatically — you do not need to do anything. The tiered context system reduces token usage by ~40% versus injecting all memory.

Memory Consolidation

An automated job runs every 4 hours. It finds groups of similar memory entries, uses Claude to synthesise each cluster into a single concise entry, and archives the originals. This keeps the memory store lean and prevents duplicates building up over time. Original entries are never hard-deleted — they are archived and can be reviewed in the Observability → Memory Health tab.

You can also trigger consolidation immediately: go to **Observability** → **Memory Health** → **Trigger Consolidation**.

Tip: write memory entries in full sentences, not just keywords. "The client's primary contact is James Brown, james@example.com, +44 7700 900000" is far more useful to an agent than "james contact".

8 — Workflows & Automation

What Are Workflows?

Workflows are multi-step AI pipelines. Each workflow chains together one or more agent calls, passing the output of one step as input to the next. They can run on a cron schedule or be triggered manually from the dashboard or via webhook from an external system.

Built-In Workflows

Workflow	What It Does	Default Schedule
Morning Briefing	Synthesises overnight memory and context into a structured daily briefing.	Mon–Fri 07:00
Memory Curation	Runs the Memory Curator Agent to deduplicate and consolidate memory entries.	Every 4 hours
Research Digest	Research Agent deep-dives a topic and produces a structured digest.	On demand
Client Report	Client Manager Agent generates status cards for all active projects.	Weekly
Analysis Run	Analysis Agent processes queued data and produces findings.	On demand
QA Sweep	QA Agent reviews recent outputs against quality and compliance rules.	On demand
Meta-Orchestrate	Meta Orchestrator decomposes a complex goal into a chain of agent calls.	On demand

Running a Workflow Manually

1. Go to **Workflows** page
2. Find the workflow you want
3. Click **Run Now**
4. The run appears in Recent Events on the Overview page

Webhook Triggers

Any workflow can be triggered by an external system (a CRM, a website form, a script) without a login token. The webhook is authenticated by a secret header.

Enable a Webhook

1. Open the Workflows page, select a workflow, click the **Webhooks** tab
2. Click **Enable Webhook** — a unique URL and HMAC secret are generated
3. Copy both the URL and the secret. The secret is shown once — store it safely.

Fire the Webhook

```
curl -X POST [http://YOUR-SERVER-IP:5000]/api/v1/workflows/morning-briefing/trigger \ -H
"X-Webhook-Secret: your-secret" \ -H "Content-Type: application/json" \ -d '{"context":
{"namespace": "global", "topic": "AI news"}}'
```

The context object is passed as input to the first pipeline step.

Viewing Workflow Run History

Each workflow card shows its last run timestamp and status. Click a workflow to expand its step-by-step run log.

9 — Client Vault

Namespaces

A namespace is a fully isolated workspace. All memory, outputs, and agent context is scoped per namespace — nothing leaks between namespaces. The Client Vault page shows:

- Namespace name and description
- Workspace statistics: number of files, total size in KB
- Number of active projects

Projects

Projects live inside a namespace and represent discrete engagements or work streams. From the Client Vault you can:

- List all projects in your namespace
- Update a project's status: **Active** / **Paused** / **Archived**
- See project descriptions and creation dates

Context Files

Context files are documents uploaded per namespace that are automatically injected into every agent system prompt for that namespace. Use this for:

- Brand guidelines and tone-of-voice documents
- Standard operating procedures
- Client briefs or background documents
- Technical specifications that agents should always be aware of

Upload a file on the Client Vault page under the **Context Files** section. Supported formats: plain text, Markdown, PDF. The file is stored in the namespace workspace and picked up on the next agent run.

Note: Keep context files concise and focused. Very large context files increase token usage on every agent call. Split large documents into smaller topical files where possible.

10 — Outputs & Reports

What Are Outputs?

Every time an agent produces a response with `save_output: true` (the default), the full output is saved to the database and the filesystem. Outputs are automatically tagged with agent name, namespace, output type, and timestamp.

Browsing Outputs

Go to the **Outputs** page. You can:

- Search by keyword across all output content (full-text)
- Filter by namespace, output type (report / summary / plan / analysis / code / other), and date range
- Click any output to expand and read the full content

Exporting an Output

Two export formats are available per output:

Format	Notes
Markdown	Raw text with Markdown formatting. Opens in any text editor or Markdown renderer. Always available.
PDF	Rendered PDF with ClaudeOS branding. Requires wkhtmltopdf installed on the server.

Bulk Delete

Select multiple outputs using the checkboxes and click the bulk delete button. This is permanent — deleted outputs are not recoverable unless Supabase cloud sync is enabled.

Tip: use the type and date filters to find old or low-quality outputs before bulk deleting to avoid accidentally removing important content.

11 — Ticketing System

Overview

The ticketing system provides a full support and task lifecycle inside ClaudeOS. Tickets can represent bugs, feature requests, client tasks, internal work items, or support requests.

Priority / SLA Tiers

Priority	Label	Meaning
P1	Critical	System down or blocking. Immediate response required.
P2	High	Major issue, significant business impact.
P3	Medium	Important but not blocking.
P4	Low	Nice-to-have, enhancement, or minor issue.

Ticket Lifecycle

Open → In Progress → Resolved → Closed

- **Open:** ticket created, not yet assigned or started
- **In Progress:** assigned and being worked on
- **Resolved:** work complete, awaiting confirmation
- **Closed:** confirmed resolved, resolution note recorded

Creating a Ticket

1. Go to **Tickets** page
2. Click **New Ticket**
3. Fill in title, description, priority, and namespace

4. Click **Create**

Assignment

- **Admin / Operator:** can assign any ticket to any user
- **Staff:** can self-assign any open ticket
- **Client / Viewer:** cannot assign tickets

Comments

Expand a ticket and click the **Comments** toggle to load threaded comments. Comments load on demand (not preloaded) to keep the page fast.

Resolution Notes

When moving a ticket to Resolved or Closed, record what was done in the resolution note field. This creates an audit trail and a knowledge base of past solutions.

Bulk Delete

Select multiple tickets with checkboxes and delete in one action. Admin / Operator only.

Stats Panel

The top of the Tickets page shows ticket counts broken down by status and priority at a glance.

12 — Quality Scoring

How It Works

Every agent run is automatically scored by Claude Haiku (a fast, low-cost AI model) on 4 dimensions. Scoring happens asynchronously in the background — it does not add any latency to the agent response. Scores typically appear 5–15 seconds after the run completes.

Scoring Dimensions

Dimension	Scale	Weight	What It Measures
Task Completion	0–5	40%	Did the output fully address the prompt? Is the task actually done?
Factual Grounding	0–5	30%	Are claims in the output grounded in the memory context that was injected? Penalises hallucination.
Conciseness	0–5	20%	Is the output the right length? Penalises padding, repetition, and unnecessary waffle.
Safety	pass/fail	10%	Does the output contain harmful, biased, or dangerous content? A fail caps the overall score at 1.0 regardless of other scores.

Overall score = (Task × 0.40) + (Factual × 0.30) + (Concise × 0.20) + (Safety × 0.10)

Where Scores Appear

- **Run History:** colour-coded pill next to each run
- **Overview:** pill on each recent event entry
- **Chat:** badge below each assistant response (~10s delay)
- **Observability** → **Quality Scores:** aggregated analytics, per-agent averages, score distribution chart

Low-Quality Threshold

Any run scoring below **2.5** is flagged as low quality. The Observability dashboard shows a red alert when any agent's rolling average drops below this threshold. Re-run with a more specific prompt if you see this.

Improving Scores

Low score on...	What to do
Task Completion	Make your prompt more specific. State exactly what you want, the format, and the desired length.
Factual Grounding	Add more relevant facts to Memory before running the agent. The agent can only ground claims in what it has been given.
Conciseness	Add "be concise" or "maximum 3 paragraphs" to your prompt. Or use an agent with lower max_tokens.
Safety	Review the output for any problematic content. Adjust the prompt to avoid triggering safety filters.

13 — Observability

The Observability page is available to **Admin** and **Operator** roles only. It has four sub-tabs.

Quality Scores Tab

- Per-agent average eval scores in a ranked table
- Score distribution chart showing how scores are spread across all runs (0.5-step buckets)
- Dimension breakdown: task completion, factual grounding, conciseness, safety
- Red alert when any agent's average drops below 2.5

Latency Tab

- p50, p95, and p99 latency across all runs (percentile-based)
- Per-agent average latency table sorted by slowest
- Time-series chart showing latency trends over the past 7 days

Expected latency ranges (claude-sonnet-4-6):

Response length	Typical latency
Short (under 200 tokens)	2–6 seconds
Medium (200–800 tokens)	6–20 seconds
Long (800+ tokens)	20–60 seconds

Token Cost Tab

- Total input and output token counts across all runs
- Estimated USD cost based on current claude-sonnet-4-6 pricing
- Per-agent breakdown: tokens used, cost, run count, average cost per run

Token cost estimates are approximate and based on list pricing at the time of the ClaudeOS release. Check Anthropic's pricing page for current rates.

Memory Health Tab

- Entry counts per namespace: total, active, consolidated, expired
- Storage size estimate per namespace
- **Trigger Consolidation** button — runs the consolidation job immediately
- Visual indicator when any namespace has not been consolidated in over 24 hours

14 — Advanced Features

Multi-Turn Conversation History

Each agent in Chat mode maintains its own conversation session. The full back-and-forth is stored in the database — not just in browser memory. This means:

- Refreshing the page does not lose the conversation (within the same session)
- The agent has access to everything said earlier in the conversation as context
- Each conversation session gets a unique ID stored in `agent_conversations`

Click **Clear Conversation** to start a fresh session when moving to a new topic — this avoids stale context polluting new requests.

Image and Screenshot Analysis

Any Chat session supports image uploads. The image is encoded as a base64 content block and sent alongside your prompt to Claude's vision API. Practical uses:

- Paste a screenshot of a report and ask the agent to summarise or critique it
- Upload a chart and ask the Analysis Agent to interpret the trends
- Send a UI screenshot to the QA Agent for a quick compliance check
- Share a document photo for text extraction and analysis

Voice Input

Requires Whisper: `pip install openai-whisper`. The model downloads on first use (~140 MB). Subsequent starts are fast (model cached in memory).

1. Click the microphone button in the Chat tab
2. Record your request (up to 30 seconds recommended)
3. ClaudeOS transcribes the audio locally — nothing is sent to any external service
4. The transcription populates the prompt field — review, edit if needed, then submit

MCP Tool Server

The Model Context Protocol (MCP) server exposes all 12 ClaudeOS agents as native tools to any MCP-compatible AI client — Claude Desktop, Cursor, or custom agents.

Starting the MCP Server

```
.\scripts\start_mcp.ps1
```

Starts on port 5100. Verify it is running:

```
curl http://localhost:5100/mcp
```

Connecting Claude Desktop

Add the following to %APPDATA%\Claude\claude_desktop_config.json:

```
{ "mcpServers": { "claudeos": { "url": "http://localhost:5100/mcp" } } }
```

Restart Claude Desktop. All 12 agents will appear as available tools in the tool picker. You can now ask Claude Desktop to "use the Writing Agent to draft a proposal" and it will call your ClaudeOS instance directly.

A2A Agent Cards

Each agent exposes a machine-readable capability card for agent-to-agent discovery:

```
GET [http://YOUR-SERVER-IP:5000]/api/v1/agents/writing-agent/.well-known/agent.json
```

The card describes the agent's name, description, input/output schema, and streaming and multi-turn capabilities. External AI orchestrators can use this endpoint to automatically discover and delegate to ClaudeOS agents.

15 — API Reference

The ClaudeOS REST API runs at `[http://YOUR-SERVER-IP:5000]/api/v1/`. All endpoints require authentication (JWT Bearer token or X-API-Key header) except `/health` and webhook trigger endpoints.

Authentication

JWT (Interactive Users)

```
# 1. Login curl -X POST [http://YOUR-SERVER-IP:5000]/api/v1/auth/login \ -H "Content-Type: application/json" \ -d '{"username":"your_user","password":"your_pass"}' # Response { "access_token": "eyJ...", "refresh_token": "eyJ...", "user": {"id": 1, "username": "your_user", "role": "operator"} } # 2. Use the access token curl [http://YOUR-SERVER-IP:5000]/api/v1/agents \ -H "Authorization: Bearer eyJ..."
```

Access tokens expire after 60 minutes. Refresh them:

```
curl -X POST [http://YOUR-SERVER-IP:5000]/api/v1/auth/refresh \ -H "Authorization: Bearer <refresh_token>"
```

API Key (Scripts and Automation)

```
curl [http://YOUR-SERVER-IP:5000]/api/v1/memory \ -H "X-API-Key: your_api_key"
```

API keys are created in the Admin Panel (admin only) and shown once on creation.

Core Endpoint Reference

Method	Endpoint	Description
POST	<code>/auth/login</code>	Login, returns access + refresh tokens
POST	<code>/auth/refresh</code>	Renew access token using refresh token
POST	<code>/auth/logout</code>	Revoke current session

GET	/auth/me	Current user info and role
GET	/health	Public health check (no auth)
GET	/memory	List memory entries (filterable)
POST	/memory	Write a memory entry
DELETE	/memory/<id>	Delete a memory entry
GET	/memory/hybrid-search	Hybrid BM25+vector search with RRF reranking
POST	/memory/consolidate	Trigger memory consolidation immediately
GET	/agents	List all registered agents
POST	/agents/<name>/run	Dispatch async agent run, returns run_id
GET	/agents/runs/<id>	Poll run status and output
GET	/agents/runs	List all runs (filterable by namespace/status)
GET	/agents/<name>/stream	SSE streaming response (token-by-token)
GET	/agents/<name>/well-known/agent.json	A2A Agent Card
GET	/outputs	List outputs
DELETE	/outputs/bulk	Bulk delete outputs
GET	/tickets	List tickets
POST	/tickets	Create a ticket
PUT	/tickets/<id>	Update ticket (status, assignee, resolution)
DELETE	/tickets/bulk	Bulk delete tickets
GET	/tickets/stats	Ticket counts by status and priority

GET	/workflows	List workflows
POST	/workflows/<name>/run	Trigger workflow manually (JWT required)
POST	/workflows/<name>/trigger	Webhook trigger (public, X-Webhook-Secret auth)
POST	/workflows/<name>/webhook/enable	Enable webhook and generate secret
GET	/system/status	System health detail

Async Run — Full Example

```
# Dispatch curl -X POST [http://YOUR-SERVER-IP:5000]/api/v1/agents/writing-agent/run \ -H
"Authorization: Bearer <token>" \ -H "Content-Type: application/json" \ -d '{ "prompt": "Write
a 1-page executive summary of Q2 results", "namespace": "my-namespace", "save_output": true }'
# Response 202 {"run_id": "abc123", "status": "pending", "poll_url":
"/api/v1/agents/runs/abc123"} # Poll until done curl
[http://YOUR-SERVER-IP:5000]/api/v1/agents/runs/abc123 \ -H "Authorization: Bearer <token>" #
Response when complete { "run_id": "abc123", "status": "done", "output": "Q2 Executive
Summary\n...", "tokens_in": 1420, "tokens_out": 380, "eval_score": 4.2 }
```

SSE Streaming — Full Example

```
curl -N
"[http://YOUR-SERVER-IP:5000]/api/v1/agents/writing-agent/stream?prompt=Hello&namespace=my-ns"
\ -H "X-API-Key: your_api_key" # Stream events arrive as: data: {"type": "token", "text":
"Hello"} data: {"type": "token", "text": " there"} data: {"type": "done", "run_id": "xyz",
"tokens_in": 900, "tokens_out": 12} # On error: data: {"type": "error", "message": "Rate limit
exceeded"}
```

16 — Security Model

Authentication

Mechanism	Details
JWT access tokens	60-minute TTL. Silently refreshed while the session is active. Never stored in plaintext — sent as a Bearer header.
Refresh tokens	7-day TTL. Stored as a SHA-256 hash in the database. Used only to renew access tokens. Revokable per-session from the Admin Panel.
API keys	For script and automation access. Created in the Admin Panel. Shown once on creation. Stored as a hash.
Webhook secrets	32-byte random hex, compared using HMAC constant-time comparison to prevent timing attacks.

Password Policy

- Minimum 10 characters
- At least one uppercase, one lowercase, one digit
- Hashed with bcrypt (12 rounds) — never stored in plaintext
- Admin can force a password change on next login per user

Account Lockout

5 consecutive failed login attempts triggers a 15-minute lockout. Configurable by admin via **Admin Panel** → **Security**. Admin can manually unlock accounts immediately from **Admin Panel** → **Users**.

Namespace Isolation

Client and Viewer role users are hard-scoped to their assigned namespace. Every API endpoint enforces namespace access in the application layer — a client user cannot read, write, or search

memory from another namespace regardless of the API parameters they send.

Audit Log

Every significant action is recorded in the audit log:

- Login success and failure
- Account lockout triggered
- Logout
- Token refresh
- User creation, deactivation, and password reset
- API key creation and revocation

The audit log is viewable in **Admin Panel** → **Audit Log**. Each entry records the event type, username, IP address, user agent, and timestamp.

Admin Panel Controls

Control	What It Does
Users	Create, deactivate, unlock, and reset passwords for all users.
API Keys	Create and revoke API keys. Raw key shown once on creation.
Sessions	View all active refresh token sessions (IP, user agent, expiry). Revoke any session immediately.
Security Config	Lockout threshold, lockout duration, access token TTL, refresh token TTL, enable/disable self-registration.
Audit Log	Paginated full event log with filtering.

What ClaudeOS Does NOT Store

- Plaintext passwords — only bcrypt hashes
- Raw refresh tokens — only SHA-256 hashes

- Raw API keys beyond the initial response — only hashes
- Anthropic API key in the database — only in the server's `.env` file

17 — Cloud Sync (Supabase)

Overview

ClaudeOS can sync outputs and memory entries to a Supabase PostgreSQL database for cloud backup and off-site storage. Sync is push-only — SQLite on your server is always the source of truth. Supabase is a mirror.

Setup

1. Create a free account at supabase.com and create a new project
2. In the Supabase SQL Editor, run the contents of `sync/supabase_schema.sql` — this creates the required tables
3. Copy your Supabase project URL and service role key from the Supabase project settings (API section)

Add to `.env`:

```
SUPABASE_URL=https://your-project.supabase.co SUPABASE_SERVICE_KEY=your_service_role_key
```

5. Restart the server — auto-sync starts immediately, firing every 15 minutes

Manual Sync

Go to **Settings** and click **Sync Now** to push immediately rather than waiting for the next scheduled run.

What Is Synced

- Memory entries
- Agent runs and outputs
- Namespaces and projects
- System events

Note: Auth data (users, sessions, passwords) is *not* synced to Supabase. It stays on your server only.

18 — Troubleshooting

Problem	Solution
Cannot reach the dashboard	Run <code>.\scripts\start.ps1</code> . Check both ports (5000 and 8501) are alive. Verify no firewall is blocking them.
Login fails	Check username and password (login is case-insensitive for username). After 5 failures, wait 15 minutes or ask admin to unlock your account.
"Must change password" shown	Enter your current password then your new password in the prompt that appears.
API returns 401 Unauthorized	Your access token has expired. Log out and log back in. If using an API key, verify it has not been revoked in the Admin Panel.
Agent runs are slow	Expected for long responses — Claude Sonnet processes ~40 tokens/second. For short tasks, use an agent with lower <code>max_tokens</code> . Check latency in Observability.
Eval scores not appearing	Scores are async — wait 10–15 seconds after the run completes. If they still don't appear, check API logs for Haiku eval errors.
Streaming returns an error	Ensure only one Flask process is running on port 5000. Restart with <code>.\scripts\start.ps1</code> .
Voice input not working	Run <code>pip install openai-whisper</code> . The first use downloads the model (~140 MB) — wait for the download to complete.
MCP server not found	Run <code>pip install mcp uvicorn</code> then <code>.\scripts\start_mcp.ps1</code> .
Runs stuck as "running"	Restart the server. Stuck runs from crashed sessions are auto-cleaned on the next health check.
Agents not responding	Check <code>ANTHROPIC_API_KEY</code> in <code>.env</code> . Verify it is not expired or rate-limited in the Anthropic console.

ChromaDB slow on first start	Normal — the sentence-transformers model loads on first request (30–60 second cold start). Subsequent requests are fast.
Ticket comments not loading	Click the Comments toggle — comments load on demand, not automatically.
Supabase sync fails	Verify <code>SUPABASE_URL</code> and <code>SUPABASE_SERVICE_KEY</code> in <code>.env</code> . Restart after any changes. Check the Settings page for error detail.
Output PDF export fails	Try Markdown export instead. PDF export may require wkhtmltopdf installed on the server.
Memory entries not appearing in agent context	Entries may be archived or expired. Check the Memory page with all filters cleared. Ensure the namespace selected when running the agent matches the namespace where you wrote the entries.

19 — Support

Contact Your Administrator

Detail	Value
Name	[ADMIN NAME]
Email	[ADMIN EMAIL]
Phone	[ADMIN PHONE]
Dashboard URL	[http://YOUR-SERVER-IP:8501]

Before Contacting Support

Please check Section 18 (Troubleshooting) first. If the issue persists:

1. Note the exact error message shown in the dashboard or browser
2. Note what you were doing when the error occurred (which page, which agent, which prompt)
3. Note whether the error is reproducible (does it happen every time?)
4. Check the Observability dashboard for any system-wide alerts

Useful Information to Provide

- Your username and role
 - Your namespace
 - The run ID (visible in Run History) if the issue is with a specific agent run
 - Approximate time the issue occurred
-